```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

data = pd.read_csv(r'D:\4th semester\Data analysis\project\Fraud.csv')

data.columns
```

```
Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg',
'newbalanceOrig',
       'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
       'isFlaggedFraud'],
      dtype='object')
```

```
data.head()
```

```
   step      type    amount     nameOrig  oldbalanceOrg
newbalanceOrig  \
0     1   PAYMENT   9839.64  C1231006815       170136.0
160296.36
1     1   PAYMENT   1864.28  C1666544295        21249.0
19384.72
2     1  TRANSFER    181.00  C1305486145          181.0
0.00
3     1  CASH_OUT    181.00   C840083671          181.0
0.00
4     1   PAYMENT  11668.14  C2048537720        41554.0
29885.86

      nameDest  oldbalanceDest  newbalanceDest  isFraud
isFlaggedFraud
0  M1979787155             0.0             0.0        0
0
1  M2044282225             0.0             0.0        0
0
2   C553264065             0.0             0.0        1
0
3    C38997010         21182.0             0.0        1
0
4  M1230701703             0.0             0.0        0
0
```

```
data.dtypes
```

```
step                int64
type               object
amount            float64
nameOrig           object
oldbalanceOrg     float64
newbalanceOrig    float64
nameDest           object
oldbalanceDest    float64
newbalanceDest    float64
isFraud             int64
isFlaggedFraud      int64
dtype: object
```

```
data.isnull().sum()
```

```
step              0
amount            0
oldbalanceOrg     0
newbalanceOrig    0
oldbalanceDest    0
newbalanceDest    0
isFraud           0
isFlaggedFraud    0
type_CASH_OUT     0
type_DEBIT        0
type_PAYMENT      0
type_TRANSFER     0
dtype: int64
```

```
data.info
```

```
<bound method DataFrame.info of          step      amount
oldbalanceOrg  newbalanceOrig  oldbalanceDest  \
0           1     9839.64      170136.00       160296.36
0.00
1           1     1864.28       21249.00        19384.72
0.00
2           1      181.00         181.00            0.00
0.00
3           1      181.00         181.00            0.00
21182.00
4           1    11668.14       41554.00        29885.86
0.00
...       ...         ...             ...             ...               .
..
6362615   743   339682.13      339682.13            0.00
0.00
6362616   743  6311409.28     6311409.28            0.00
0.00
6362617   743  6311409.28     6311409.28            0.00
```

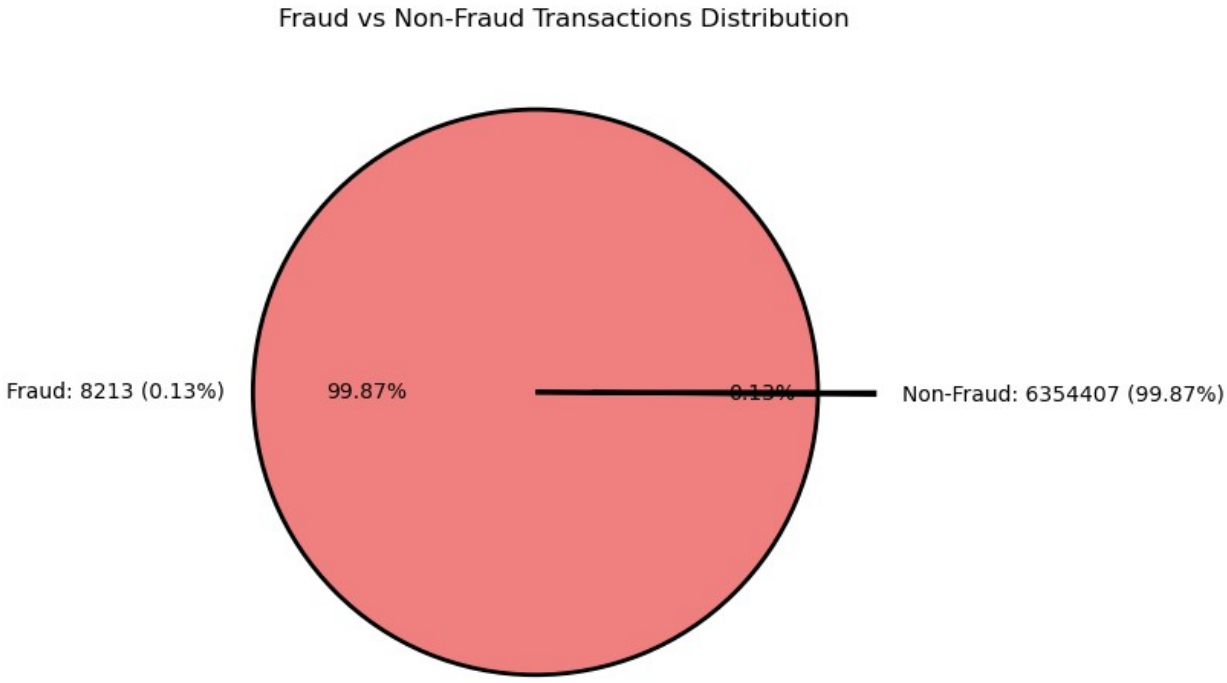
```
68488.84
6362618   743   850002.52      850002.52            0.00
0.00
6362619   743   850002.52      850002.52            0.00
6510099.11

         newbalanceDest  isFraud  isFlaggedFraud  type_CASH_OUT
type_DEBIT  \
0                  0.00        0               0          False
False
1                  0.00        0               0          False
False
2                  0.00        1               0          False
False
3                  0.00        1               0           True
False
4                  0.00        0               0          False
False
...                 ...      ...             ...            ...
...
6362615       339682.13        1               0           True
False
6362616            0.00        1               0          False
False
6362617      6379898.11        1               0           True
False
6362618            0.00        1               0          False
False
6362619      7360101.63        1               0           True
False

         type_PAYMENT  type_TRANSFER
0                True          False
1                True          False
2               False           True
3               False          False
4                True          False
...               ...            ...
6362615         False          False
6362616         False           True
6362617         False          False
6362618         False           True
6362619         False          False

[6362620 rows x 12 columns]>

data.describe().T

                    count          mean           std  min
25%  \
```

```
step            6362620.0  2.433972e+02  1.423320e+02  1.0    156.00

amount          6362620.0  1.798619e+05  6.038582e+05  0.0  13389.57

oldbalanceOrg   6362620.0  8.338831e+05  2.888243e+06  0.0      0.00

newbalanceOrig  6362620.0  8.551137e+05  2.924049e+06  0.0      0.00

oldbalanceDest  6362620.0  1.100702e+06  3.399180e+06  0.0      0.00

newbalanceDest  6362620.0  1.224996e+06  3.674129e+06  0.0      0.00

isFraud         6362620.0  1.290820e-03  3.590480e-02  0.0      0.00

isFlaggedFraud  6362620.0  2.514687e-06  1.585775e-03  0.0      0.00


                        50%           75%           max
step                239.000  3.350000e+02  7.430000e+02
amount            74871.940  2.087215e+05  9.244552e+07
oldbalanceOrg     14208.000  1.073152e+05  5.958504e+07
newbalanceOrig        0.000  1.442584e+05  4.958504e+07
oldbalanceDest   132705.665  9.430367e+05  3.560159e+08
newbalanceDest   214661.440  1.111909e+06  3.561793e+08
isFraud               0.000  0.000000e+00  1.000000e+00
isFlaggedFraud        0.000  0.000000e+00  1.000000e+00
```

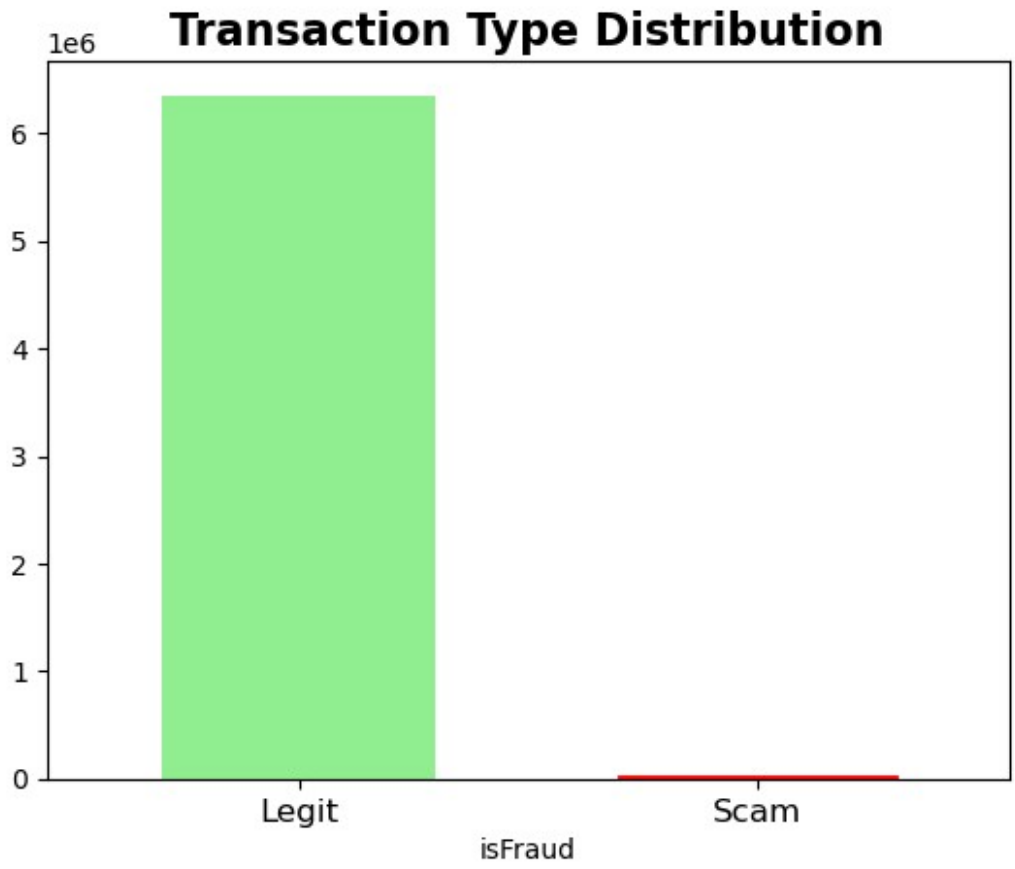
```
data['isFraud'].value_counts()
```

```
isFraud
0    6354407
1       8213
Name: count, dtype: int64
```

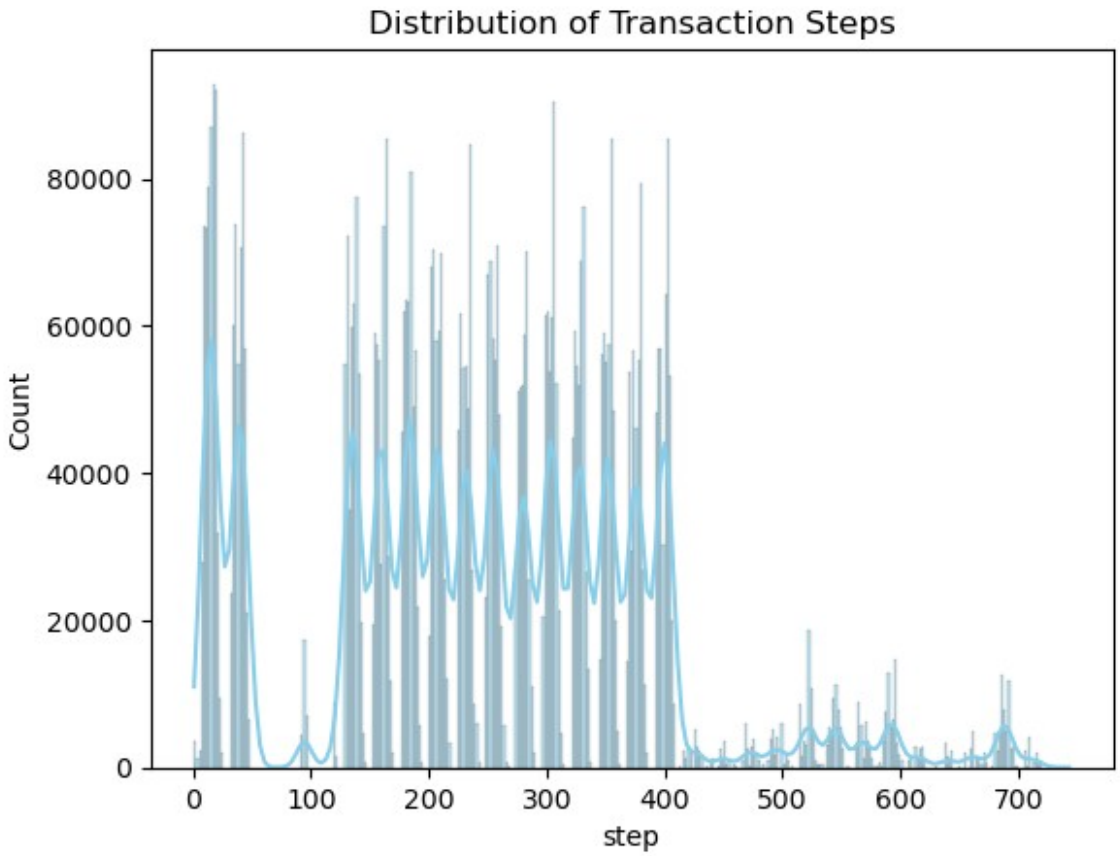
```
plt.figure(figsize=(10,6))
sizes = data['isFraud'].value_counts()
labels = [f'Fraud: {sizes[1]} ({sizes[1] / sum(sizes) * 100:.2f}%)',
          f'Non-Fraud: {sizes[0]} ({sizes[0] / sum(sizes) * 100:.2f}
%)']
explode = [0.1, 0.1]
color = ['lightcoral', 'red']
plt.pie(sizes, explode=explode, labels=labels, autopct="%1.2f%%",
colors=color,wedgeprops={'edgecolor': 'black', 'linewidth': 2})
plt.title("Fraud vs Non-Fraud Transactions Distribution")
plt.show()
```



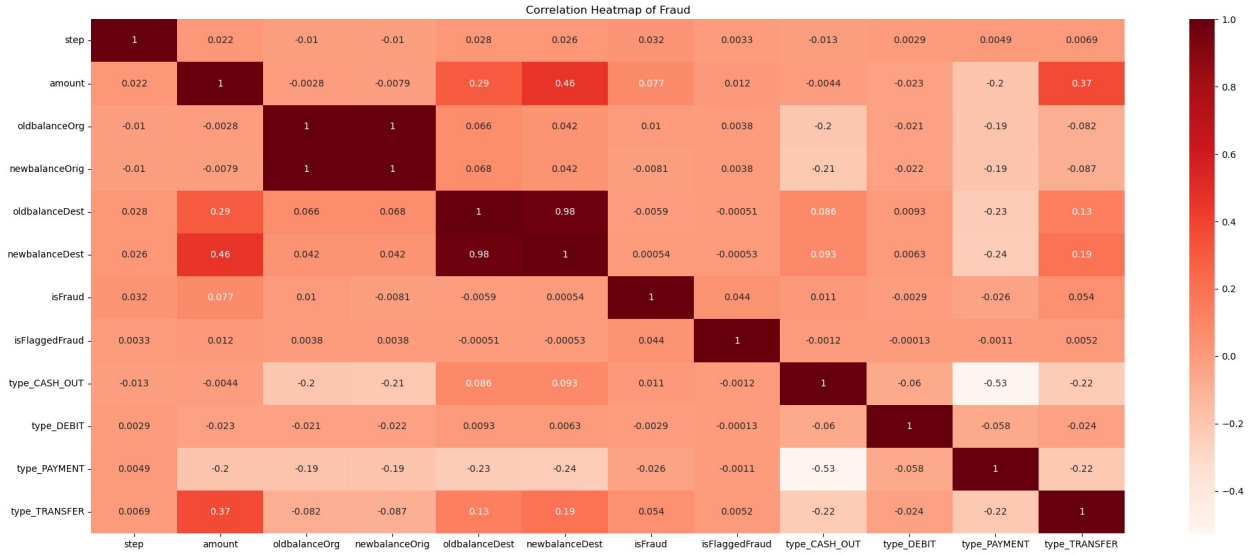

```
# Count the occurrences of each class (fraud and non-fraud)
count_classes = data['isFraud'].value_counts()
ax = count_classes.plot(kind='bar', rot=0, color=['lightgreen',
'salmon'], width=0.6)
plt.title('Transaction Type Distribution', fontsize=16,
fontweight='bold')
plt.xticks(range(2), ['Legit', 'Scam'], fontsize=12)

# Highlight fraud (Scam),thicker border for fraud bar
for i in range(2):
    if i == 1:
        ax.patches[i].set_edgecolor('red')
        ax.patches[i].set_linewidth(2)
plt.show()
```




```
sns.histplot(data['step'], kde=True, color='skyblue')
plt.title("Distribution of Transaction Steps")
plt.show()
```




```
corr_matrix = data.corr()
plt.figure(figsize=(25, 10))
sns.heatmap(corr_matrix, annot=True, cmap='Reds')
plt.title("Correlation Heatmap of Fraud")
plt.show()
```



```
# Define feature columns (X) and target column (y)
X = data.iloc[:, 0:-1]  # All columns except the last one
y = data.iloc[:, -1]    # The last column (target variable)
```

```
X.shape
```

```
(6362620, 11)
```

```
y.shape
```

```
(6362620,)
```

```
# Split data into training and testing sets (75% train, 25% test)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.25, random_state=42)
```

```
X_train.shape
```

```
(4771965, 11)
```

```
X_test.shape
```

```
(1590655, 11)
```

```
y_train.shape
```

```
(4771965,)
```

```
y_test.shape
```

```
(1590655,)
```

```
logit = LogisticRegression(max_iter=1000)  # max_iter ensures
convergence if needed

# Fit the model on the training data
logit.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=1000)
```

```
# Predict on training and test data
y_pred_train = logit.predict(X_train)
y_pred_test = logit.predict(X_test)

#confusion Matrix
print("Confusion Matrix (Train):")
print(confusion_matrix(y_train, y_pred_train))
print("Confusion Matrix (Test):")
print(confusion_matrix(y_test, y_pred_test))
```

```
Confusion Matrix (Train):
[[4172634  199663]
 [ 225589  174079]]
```

```
Confusion Matrix (Test):
[[1391759   65655]
 [  75383   57858]]
```

```
#classification Report
print("\nClassification Report (Train):")
print(classification_report(y_train, y_pred_train))
print("Classification Report (Test):")
print(classification_report(y_test, y_pred_test))
```

```
Classification Report (Train):
              precision    recall  f1-score   support

       False       0.95      0.95      0.95   4372297
        True       0.47      0.44      0.45    399668

    accuracy                           0.91   4771965
   macro avg       0.71      0.69      0.70   4771965
weighted avg       0.91      0.91      0.91   4771965

Classification Report (Test):
              precision    recall  f1-score   support

       False       0.95      0.95      0.95   1457414
        True       0.47      0.43      0.45    133241

    accuracy                           0.91   1590655
   macro avg       0.71      0.69      0.70   1590655
weighted avg       0.91      0.91      0.91   1590655
```

```
#accuracy Score
print("\nAccuracy (Train):", accuracy_score(y_train, y_pred_train))
print("Accuracy (Test):", accuracy_score(y_test, y_pred_test))
```

```
Accuracy (Train): 0.9108853480694011
Accuracy (Test): 0.9113333815315074
```